7.4. Summary

Following from the lock-based data structures of chapter 6, this chapter has described simple implementations of various lock-free data structures, starting with a stack and a queue, as before. You saw how you must take care with the memory ordering on your atomic operations to ensure that there are no data races and that each thread sees a coherent view of the data structure. You also saw how memory management becomes much harder for lock-free data structures than lock-based ones and examined a couple of mechanisms for handling it. You also saw how to avoid creating wait loops by helping the thread you're waiting for to complete its operation.

Designing lock-free data structures is a difficult task, and it's easy to make mistakes, but such data structures have scalability properties that are important in some situations. Hopefully, by following through the examples in this chapter and reading the guidelines, you'll be better equipped to design your own lock-free data structure, implement one from a research paper, or find the bug in the one your former colleague wrote just before he left the company.

Wherever data is shared between threads, you need to think about the data structures used and how the data is synchronized between threads. By designing data structures for concurrency, you can encapsulate that responsibility in the data structure itself, so the rest of the code can focus on the task it's trying to perform *with* the data rather than the data synchronization. You'll see this in action in chapter 8 as we move on from concurrent data structures to concurrent code in general. Parallel algorithms use multiple threads to improve their performance, and the choice of concurrent data structure is crucial where the algorithms need their worker threads to share data.